



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 11

по курсу «Функциональное и логическое программирование»

Студент ИУ7-62Б
(Группа)

(Подпись, дата)

А. Н. Диваев
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Н. Б. Толпинская
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю. В. Строганов
(И. О. Фамилия)

1 Задание

Используя хвостовую рекурсию, разработать эффективную программу, позволяющую:

- найти длину списка (по верхнему уровню);
- найти сумму элементов числового списка;
- найти сумму элементов числового списка, стоящих на нечетных позициях исходного списка (нумерация от 0);
- сформировать список из элементов числового списка, больших заданного значения;
- удалить заданный элемент из списка (один или все вхождения).
- объединить два списка.

Убедиться в правильности результатов. Для одного из вариантов вопроса уметь составить таблицу, отражающую конкретный порядок работы системы.

2 Программа

2.1 База знаний

Листинг 1 – Исходный код базы знаний

```
1 len(List, Length) :-
2     len_rec(List, 0, Length).
3
4 len_rec([], Acc, Acc) :- !.
5 len_rec(_|Tail, Acc, Length) :-
6     NewAcc is Acc + 1,
7     len_rec(Tail, NewAcc, Length).
8
9
10 sum(List, Sum) :-
11     sum_rec(List, 0, Sum).
12
13 sum_rec([], Acc, Acc) :- !.
14 sum_rec([Head|Tail], Acc, Sum) :-
15     NewAcc is Acc + Head,
16     sum_rec(Tail, NewAcc, Sum).
17
18
19 sum_odd_pos(List, Sum) :-
20     sum_odd_pos_rec(List, 0, Sum).
21
22 sum_odd_pos_rec([], Acc, Acc) :- !.
23 sum_odd_pos_rec([_], Acc, Acc) :- !.
24 sum_odd_pos_rec([_, Odd | Tail], Acc, Sum) :-
25     NewAcc is Acc + Odd,
26     sum_odd_pos_rec(Tail, NewAcc, Sum).
27
28
29 filter_gt(List, V, Result) :-
30     filter_gt_rec(List, V, [], Result).
31
32 filter_gt_rec([], _, Acc, Acc) :- !.
33 filter_gt_rec([Head|Tail], V, Acc, Result) :-
34     Head > V, !,
35     filter_gt_rec(Tail, V, [Head|Acc], Result).
36 filter_gt_rec([_|Tail], V, Acc, Result) :-
37     filter_gt_rec(Tail, V, Acc, Result).
38
39
```

Листинг 1 (продолжение)

```
40 remove(List, Element, Result) :-  
41     remove_rec(List, Element, [], Result).  
42  
43 remove_rec([], _, Acc, Acc) :- !.  
44 remove_rec([Element|Tail], Element, Acc, Result) :- !,  
45     remove_rec(Tail, Element, Acc, Result).  
46 remove_rec([Head|Tail], Element, Acc, Result) :-  
47     remove_rec(Tail, Element, [Head|Acc], Result).  
48  
49  
50 merge(List1, List2, Result) :-  
51     prepend(List1, List2, Result).  
52  
53 prepend([], AccList, AccList) :- !.  
54 prepend([Head|Tail], AccList, Result) :-  
55     prepend(Tail, [Head|AccList], Result).
```

2.2 Вопросы задания

Для получения длины списка необходимо задать вопрос: существует ли такой Результат при конкретизированном Списке, для которого удовлетворяется отношение `len`.

Листинг 2 – Нахождение длины списка

```
1 ?- len([a, b, c, d], L).
```

Для получения суммы элементов числового списка необходимо задать вопрос: существует ли такой Результат при конкретизированном числовом Списке, для которого удовлетворяется отношение `sum`.

Листинг 3 – Нахождение суммы элементов списка

```
1 ?- sum([1, 2, 3, 4], S).
```

Для получения суммы элементов числового списка, стоящих на нечетных позициях (нумерация с 0), необходимо задать вопрос: существует ли такой Результат при конкретизированном числовом Списке, для которого удовлетворяется отношение `sum_odd_pos`.

Листинг 4 – Сумма элементов на нечетных позициях

```
1 ?- sum_odd_pos([10, 20, 30, 40, 50], S).
```

Для формирования списка из элементов числового списка, больших заданного значения, необходимо задать вопрос: существует ли такой результирующий Список при конкретизированных исходном числовом Списке и пороговом Значении, для которых удовлетворяется отношение `filter_gt`.

Листинг 5 – Фильтрация списка по пороговому значению

```
1 ?- filter_gt([2, 8, 3, 10, 5], 5, Res).
```

Для удаления заданного элемента из списка необходимо задать вопрос: существует ли такой результирующий Список при конкретизированных исходном Списке и удаляемом Элементе, для которых удовлетворяется отношение `remove`.

Листинг 6 – Удаление заданного элемента из списка

```
1 ?- remove([1, 3, 2, 3, 4], 3, Res).
```

Для объединения двух списков в один результирующий необходимо задать вопрос: существует ли такой результирующий Список при конкретизированных первом Списке и втором Списке, для которых удовлетворяется отношение `merge`.

Листинг 7 – Объединение двух списков

```
1 ?- merge([1, 2], [3, 4], Res).
```

3 Тестирование

Вопросы к системе представлены в листингах 8–13.

Листинг 8 – Тестирование отношения len

```
1 ?- len([], L).
2 L = 0.
3
4 ?- len([a], L).
5 L = 1.
6
7 ?- len([a, 6, 9, d, 3], L).
8 L = 5.
```

Листинг 9 – Тестирование отношения sum

```
1 ?- sum([], S).
2 S = 0.
3
4 ?- sum([52], S).
5 S = 52.
6
7 ?- sum([10, -5, 20, -3], S).
8 S = 22.
```

Листинг 10 – Тестирование отношения sum_odd_pos

```
1 ?- sum_odd_pos([10], S).
2 S = 0.
3
4 ?- sum_odd_pos([10, 20, 30, 40], S).
5 S = 60.
6
7 ?- sum_odd_pos([10, 20, 30, 40, 50], S).
8 S = 60.
```

Листинг 11 – Тестирование отношения filter_gt

```
1 ?- filter_gt([2, 5, 8], 10, Result).
2 Result = [].
3
4 ?- filter_gt([10, 20, 30], 5, Result).
```

Листинг 11 (продолжение)

```
5 Result = [30, 20, 10].  
6  
7 ?- filter_gt([2, 8, 3, 10, 5], 5, Result).  
8 Result = [10, 8].
```

Листинг 12 – Тестирование отношения remove

```
1 ?- remove([1, 2, 3], 5, Result).  
2 Result = [3, 2, 1].  
3  
4 ?- remove([3, 3, 3], 3, Result).  
5 Result = [].  
6  
7 ?- remove([1, 3, 2, 3, 4], 3, Result).  
8 Result = [4, 2, 1].
```

Листинг 13 – Тестирование отношения merge

```
1 ?- merge([], [3, 4], Result).  
2 Result = [3, 4].  
3  
4 ?- merge([1, 2], [], Result).  
5 Result = [2, 1].  
6  
7 ?- merge([3, 1], [3, 7], Result).  
8 Result = [1, 3, 3, 7].
```

4 Порядок работы системы

Порядок работы системы указан в таблице 1.

Таблица 1 – Порядок работы системы для запроса $\text{len}([a, b], \text{Res})$

№ шага	Состояние резольвенты, и вывод: дальнейшие действия (почему?)	Для каких термов запускается алгоритм унификации: $T1=T2$ и каков результат (и подстановка)	Дальнейшие действия: прямой ход или откат (почему и к чему приводит?)
0	Резольвента непустая: $\text{len}([a, b], \text{Res})$. Прямой ход.		Запуск алгоритма унификации для верхней подцели с 1-ым заголовком из БЗ ($\text{len_rec}/3$).
1	$\text{len}([a, b], \text{Res})$. Унификация неуспешна.	$\text{len}([a, b], \text{Res}) = \text{len_rec}([], \text{Len}, \text{Len})$ Результат: Неудача (разные функции).	Переход к следующему предложению в БЗ (со 2-м заголовком $\text{len_rec}/3$).
2	$\text{len}([a, b], \text{Res})$. Унификация неуспешна.	$\text{len}([a, b], \text{Res}) = \text{len_rec}([_ \text{Rest}], \text{Len}, \text{Res})$ Результат: Неудача (разные функции).	Переход к следующему предложению в БЗ ($\text{len}/2$).

Продолжение таблицы 1

№ шага	Состояние резольвенты, и вывод: дальнейшие действия (почему?)	Для каких термов запускается алгоритм унификации: $T1=T2$ и каков результат (и подстановка)	Дальнейшие действия: прямой ход или откат (почему и к чему приводит?)
3	<code>len([a, b], Res)</code> . Успешная унификация. Редукция подцели: <code>len_rec([a, b], 0, Res)</code> .	<code>len([a, b], Res) = len(List, Res)</code> Результат: Успех. Подстановка: <code>{List = [a, b], Res = Res}</code>	Выполнение редукции подцели и применение подстановки к новому состоянию резольвенты. Прямой ход. Запуск алгоритма унификации для новой верхней подцели.
4	<code>len_rec([a, b], 0, Res)</code> . Унификация неуспешна.	<code>len_rec([a, b], 0, Res) = len_rec([], Len, Len)</code> Результат: Неудача (список <code>[a, b]</code> и пустой список <code>[]</code> не сопоставляются).	Переход к следующему предложению в БЗ (со 2-м заголовком <code>len_rec/3</code>).
5	<code>len_rec([a, b], 0, Res)</code> . Успешная унификация. Редукция подцели: <code>is(NewLen, +(0, 1))</code> , <code>len_rec(Rest, NewLen, Res)</code> .	<code>len_rec([a, b], 0, Res) = len_rec([_ Rest], Len, Res)</code> Результат: Успех. Подстановка: <code>{Rest = [b], Len = 0}</code>	Выполнение редукции подцели и применение подстановки. Прямой ход. Вычисление встроенного предиката <code>is</code> . <code>{NewLen = 1}</code> .

Продолжение таблицы 1

№ шага	Состояние резольвенты, и вывод: дальнейшие действия (почему?)	Для каких термов запускается алгоритм унификации: $T1=T2$ и каков результат (и подстановка)	Дальнейшие действия: прямой ход или откат (почему и к чему приводит?)
6	<code>len_rec([b], 1, Res)</code> . Унификация неуспешна.	<code>len_rec([b], 1, Res) = len_rec([], Len, Len)</code> Результат: Неудача (несопоставимые списки).	Переход к следующему предложению в БЗ (со 2-м заголовком <code>len_rec/3</code>).
7	<code>len_rec([b], 1, Res)</code> . Успешная унификация. Редукция подцели: <code>is(NewLen, +(1, 1))</code> , <code>len_rec(Rest, NewLen, Res)</code> .	<code>len_rec([b], 1, Res) = len_rec([_ Rest], Len, Res)</code> Результат: Успех. Подстановка: <code>{Rest = [], Len = 1}</code>	Выполнение редукции подцели и применение подстановки. Прямой ход. Вычисление встроенного предиката <code>is</code> . <code>{NewLen = 2}</code> .
8	<code>len_rec([], 2, Res)</code> . Успешная унификация (базовый случай). Редукция подцели: <code>!</code> .	<code>len_rec([], 2, Res) = len_rec([], Len, Len)</code> Результат: Успех. Подстановка: <code>{Len = 2, Res = 2}</code>	Выполнение редукции подцели и применение подстановки к новому состоянию резольвенты. Прямой ход. Срабатывает предикат отсечения (<code>!</code>).

Продолжение таблицы 1

№ шага	Состояние резольвенты, и вывод: дальнейшие действия (почему?)	Для каких термов запускается алгоритм унификации: $T1=T2$ и каков результат (и подстановка)	Дальнейшие действия: прямой ход или откат (почему и к чему приводит?)
9	!. Предикат отсечения.	! Результат: Истина.	Резольвента пуста. Знания подобраны: {Res = 2}. Завершение работы.